2026

PROG7312 POE

SMART-X REPORT

MIHIR JAGDAW

ST10449392 | st10449392@myemeris.edu.za

Contents

Implementation Report 3

Data Structure Explanations 3

Self-balancing tree: AVL tree with range queries 3

Graph, breadth-first search and depth-first search 4

Minimum Spanning Tree: Prim & Kruskal for power-efficient routing 5

Cheapest routes: Dijkstra 6

Self-healing 7

Scaling 7

History tree 7

Mesh analysis 8

Whole system under load 8

Testing 9

Project Completion Report 10

Completed Requirements 10

Challenges and their resolutions 11

A bug only found live 11

Backbone was not always the cheapest route 11

Cause listed below the effects 11

A disk leak caused passing tests 11

Dependency cycle from Part 2 feedback 11

What I learnt 12

A MST and a shortest path are two different things 12

A passing test only counts if it can fail 12

Unit tests cannot see the platform 12

Small interfaces break dependency cycles 12

Technology Recommendations 12

Ingest over MQTT with per-device certificates 12
Recommendation 12
Justification 12
Push changes to the browser with SignalR 13
Recommendation 13
Justification 13
.NET fit 13
Partitioned time-series storage, with the AVL tree as a hot cache 13
Recommendation 13
Justification 13
.NET fit 14
Authenticate and authorise the API 14
Recommendation 14
Justification 14
.NET fit 14
Observability and automated checks (OpenTelemetry and CI) 14
Recommendation 14
Justification 15
.NET fit 15
References 16

Implementation Report

Data Structure Explanations

Self-balancing tree: AVL tree with range queries

What it stores

The tree stores every reading sent by a device, keyed by a timestamp, one tree per device (AvlTree.cs, TelemetryArchive.cs). The historical log asks questions like, “readings 26 to 50 between 14:00 and 15:00”.

Why self-balancing?

Timestamps arrive in increasing order, which turns a plain binary search tree into a linked list with O(n) lookups. An AVL tree keeps the two subtrees of every node within one level of each other, rotating after each insert or removal. I chose AVL over a red-black tree because its tighter balance makes lookups and range queries faster, which suits a log that is read far more than it is changed

How the range queries work

Each node also stores the size of its subtree. Counting the keys below a timestamp then skips whole left subtrees without visiting them, so the number of readings in a window is two O(log n) counts subtracted, and a page can start at any position without walking past the earlier ones:

```
public int CountLess(TKey key)             // keys strictly less than "key"
{
    var count = 0;
    for (var node = _root; node is not null;)
    {
        if (key.CompareTo(node.Key) <= 0) node = node.Left;
        else { count += 1 + SizeOf(node.Left); node = node.Right; }   // skip the whole left
subtree
    }
    return count;
}

// One page of the readings between from and to: find the start in O(log n), then walk on in
order.
public IEnumerable<KeyValuePair<TKey, TValue>> Between(TKey from, TKey to, int skip, int take)
{
    var inRange = CountBetween(from, to);
    return Slice(CountLess(from) + skip, Math.Min(take, inRange - skip));
}
```

The balance is repaired on the way back up from every insert and removal:

```
private static Node Rebalance(Node node)
{
    Refresh(node);                                  // height and size from the children
    var lean = HeightOf(node.Left) - HeightOf(node.Right);
    if (lean > 1)                                   // left side too tall
    {
        if (HeightOf(node.Left!.Left) < HeightOf(node.Left.Right)) node.Left =
RotateLeft(node.Left);
        return RotateRight(node);
    }
    if (lean < -1)                                  // right side too tall
    {
        if (HeightOf(node.Right!.Right) < HeightOf(node.Right.Left)) node.Right =
RotateRight(node.Right);
        return RotateLeft(node);
    }
    return node;
}
```

Graph, breadth-first search and depth-first search

The mesh is an undirected weighted graph stored as an adjacency list (MeshGraph.cs, GraphTraversal.cs). A mesh is sparse, since each device reaches only a handful of neighbours, so a list needs O(V + E) memory where a matrix would need O(V²). Vertices are the devices plus one gateway; an edge is a radio link whose weight is its energy cost.

```
while (queue.Count > 0)                         // breadth-first search
{
    var current = queue.Dequeue();
    order.Add(current);
    foreach (var (neighbour, _) in graph.Neighbors(current))
    {
        if (hops.ContainsKey(neighbour)) continue;   // already seen: the hops entry doubles
as the visited marker
        hops[neighbour] = hops[current] + 1;
        parent[neighbour] = current;
        queue.Enqueue(neighbour);
    }
}
```

- **BFS** (a Queue) finds the fewest hops from a device to every other device. It answers "who is connected?", traces connections from the gateway or from any device, and finds devices cut off when relays fail.
- **DFS** uses an explicit Stack instead of recursion, so a chain of 20,000 devices cannot overflow the call stack (a test builds exactly that chain).
- **Connected components** run BFS from each unvisited device. They find islands of devices that cannot hear each other, so the builder can join them.

Minimum Spanning Tree: Prim & Kruskal for power-efficient routing

A minimum spanning tree connects every device to the gateway with the cheapest possible set of links and no loops. Two algorithms are implemented and both run on every analysis:

- **Prim**, grown outwards from the gateway with a PriorityQueue as the heap: O(E log V). It builds the backbone the map shows.
- **Kruskal** sorts the links and keeps each one that joins two groups not yet connected, using union-find with path compression and union by rank: O(E log E). It verifies Prim: if the totals ever disagree, the API logs a warning and the page's green "Prim and Kruskal agree" badge turns red.

```
// Prim: always take the cheapest link that leaves the tree built so far.
while (frontier.TryDequeue(out var edge, out _))
{
    if (!inTree.Add(edge.B)) continue;   // the far end joined through a cheaper link since
this one was queued
    edges.Add(edge);
    total += edge.Weight;
    AddLinksFrom(edge.B);
}

// Kruskal (abridged): cheapest first; keep an edge only if it joins two separate groups, so
it never closes a loop.
foreach (var edge in graph.Edges.OrderBy(e => e.Weight))
{
    int a = Find(index[edge.A]), b = Find(index[edge.B]);
    if (a == b) continue;
    /* union by rank */
    edges.Add(edge);
    total += edge.Weight;
}
```

As a result, on a 40-device demo estate, the backbone costs 10,947 nJ/bit against 51,685 for using every link, saving 79%.

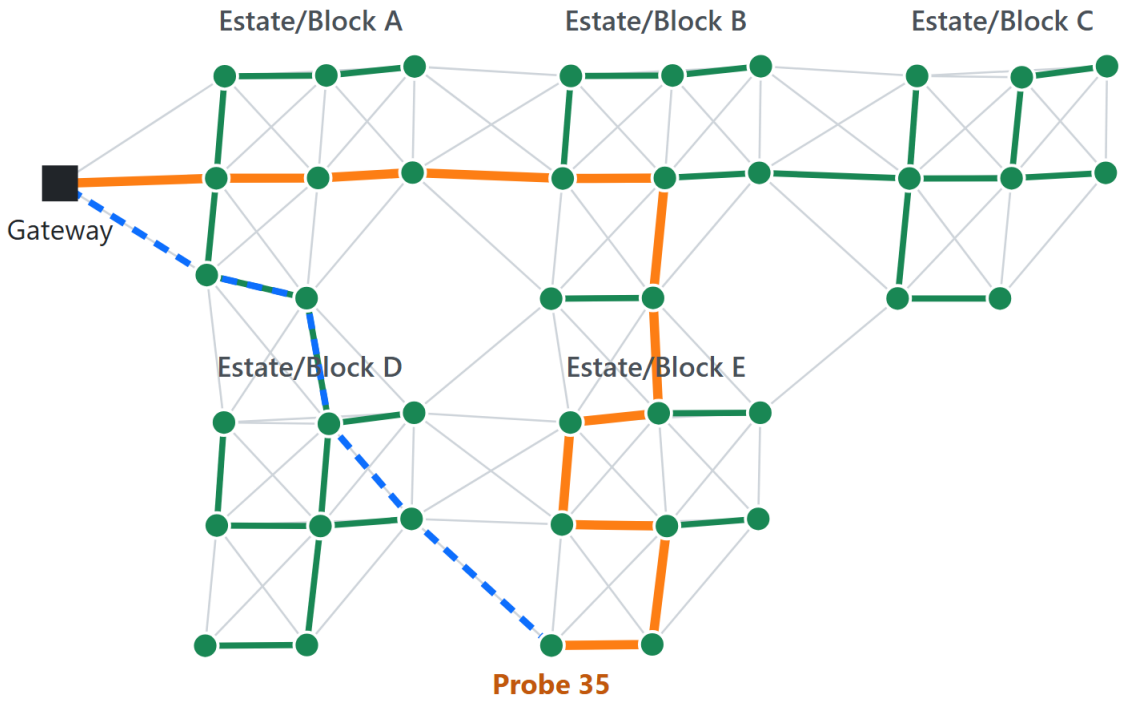


Figure 1: Demo Estate drawn by the API

However, the trade-off is that the tree minimises the cost of the whole network and not every device's own route. The cost of collecting a reading from every device along the backbone would cost almost 20% more than along each device's cheapest route and Probe 35 takes 12 hops where 5 would do, hence why Dijkstra is also used.

Cheapest routes: Dijkstra

Dijkstra's algorithm was used from the gateway to give every device its cheapest possible route, with a PriorityQueue. The route tracker draws it next to the backbone route, and it finds the backup route (remove the device's first hop from the graph and run it again). A PriorityQueue cannot lower an entry's priority, so a cheaper route queues the device again and the stale entry is skipped:

```
while (queue.TryDequeue(out var current, out var cost))
{
    if (cost > distance[current]) continue;           // stale entry: a cheaper route was
found already
    foreach (var (neighbour, weight) in graph.Neighbors(current))
    {
        var viaCurrent = cost + weight;
        if (distance.TryGetValue(neighbour, out var best) && viaCurrent >= best) continue;
        distance[neighbour] = viaCurrent;
        parent[neighbour] = current;
        queue.Enqueue(neighbour, viaCurrent);
    }
}
```

Self-healing

A device is down while it has an active Disconnected alert (silent for 15 seconds). The analyzer removes down devices from the graph and reruns BFS, Prim, Kruskal and Dijkstra, and a device whose every route ran through down devices is shown as *Isolated*. Comparing the new result with the previous one writes the self-healing log (Down, Up, CutOff, Reconnected, Rerouted). The analysis is cached and recomputed only when the set of down devices changes, so requests read the stored result.

Scaling

History tree

The readings in a window of about 100 readings at a random position, 5,000 queries (benchmark-history.cs).

Readings stored	Tree height (AVL limit)	AVL tree	Sorted array + binary search	Unsorted list scan	AVL vs scan
1,000	10 (15)	7.1 µs	1.1 µs	6.8 µs	1×
10,000	14 (20)	7.2 µs	1.0 µs	58.0 µs	8×
100,000	17 (25)	6.7 µs	1.0 µs	479.2 µs	72×
1,000,000	20 (29)	6.1 µs	2.9 µs	5,829.5 µs	957×

The tree's cost barely changes as the history grows a thousand-fold; a scan grows in proportion. A sorted array is faster for a history that never changes, but the tree wins where this system needs it to: out-of-order inserts (200,000: AVL 250 ms, sorted array 2,015 ms), removal for retention, and counting (a window holding half of a million-reading history: 0.2 µs against 6,486 µs for a scan). Live, a two-hour window out of up to 259,212 readings took 17 to 220 µs on the server.

Mesh analysis

Devices	Links	Build the mesh	Analyse (BFS, Prim, Kruskal, Dijkstra)	Fail a device, write the log
200	828	1 ms	2 ms	2 ms
1,000	4,231	14 ms	16 ms	15 ms
2,000	9,311	49 ms	44 ms	48 ms

On a 20,000-vertex graph the traversals, Prim, Kruskal and Dijkstra each finish in 81 to 285 ms. Building links compares every pair of devices, so it is $O(n^2)$.

Whole system under load

Each device reports every 5 seconds (loadtest-mesh.cs).

	200 devices	1,000 devices
GET api/Topology	168 KB (25.5 KB compressed)	816 KB (130 KB compressed)
Ingest, default 100 ms delay	falls behind: ~10 readings/s handled against 40/s sent	not sustainable
Ingest, delay set to 0	5,000-reading burst absorbed at 850 to 1,375/s	keeps up (~200 readings/s)

Retention: each device adds about 1 MB of memory (56 bytes a reading) and 2.4 MB of disk (146 bytes a row) per day, so large fleets need a shorter History:RetentionDays.

Testing

Script	Checks	What it covers
test-part3-graph.cs	48	Graph operations; BFS, DFS and components against an independent Floyd–Warshall calculation; Prim and Kruskal against each other and a brute-force minimum; Dijkstra; a 20,000-vertex run
test-part3-avl.cs	27	Balance, heights and sizes valid after 30,000 random adds and removes checked against a SortedDictionary; 10,000 random windows and 2,000 random pages against plain filters; concurrent readers and writers; 1,000,000 readings
test-part3-mesh.cs	32	Layout and subnet rules; the radio-cost crossover; connectivity for 0 to 200 devices; ring and chain failures; the self-healing log; route and backup route; 2,000 devices

The tests were themselves tested by planting bugs and checking that a test fails: 8 broken graph variants, 7 in the AVL tree and 7 in the mesh analyzer (two survived at first and led to stronger tests, see the Completion Report).

Project Completion Report

Completed Requirements

Requirement	Status	Where
Topology dashboard visualising the mesh	Complete	Topology page and live SVG mesh map: status colours, backbone, subnet filter, self-healing log
Track a sensor's routing path by MAC	Complete	Route tracker: backbone route, cheapest route, backup route, neighbours (GET api/Topology/route/{mac})
Historical telemetry log for a sensor	Complete	History panel with time window, paging and chart (GET api/History/{mac})
Self-balancing tree with range queries	Complete	AVL tree with subtree sizes, one per device
BFS/DFS and MST (Prim, Kruskal) for power-efficient routing	Complete	Graph classes and the topology service; Dijkstra added as the optional extra
Menu integration of all three modules	Complete	Menu badge counts down devices; *Show on mesh map* and *Open in command stream* link the modules; state survives navigation
README, report, recommendations, feedback ledger, Docker files	Complete	README.md, this document, FEEDBACK_UPDATES.md, docker-compose.yml

Challenges and their resolutions

A bug only found live

At startup all in-memory tests passed, but the API crashed because EF Core couldn't translate a DateTimeOffset comparison on SQLite, because it stores it as text. To ofix this I replaced it with a SQL delete on the UTC text, which sorts correctly.

Backbone was not always the cheapest route

When live, collecting along the backbone would cost 17%-30% more than along each device's cheapest route and on device took 11 hops where it could do it in 5. The reason for this was due to a MST minimising the whole network. I added Dijkstra to compare.

Cause listed below the effects

A live run showed a failure that displayed "Rerouted" lines above "Probe 16 went down", due to a tick's events being queued in order but read newest first. I now queue each tick's events backwards and added a test, as before there wasn't a test that checked the order.

A disk leak caused passing tests

After a run the SQLite log file was 384 MB beside a 331 KB database. 13 KB per reading were never folded back. Turning off connection pooling fixed it but cut writes from 1198 per second to just 182. Instead, I ran an explicit checkpoint every 1000 readings.

Dependency cycle from Part 2 feedback

To make learning asynchronous, the alert tracker would need the prediction service, which needs the insight engine, which needs the tracker. So I split "record an action" (a channel behind a one-method interface) from "learn from actions" (the background service), so the tracker depends only on the channel.

What I learnt

A MST and a shortest path are two different things

The MST minimises the cost of the whole network, whilst the shortest path is the cost of a route, which are both needed by real mesh views.

A passing test only counts if it can fail

Planting bugs allowed me to expose weak tests that clean runs couldn't.

Unit tests cannot see the platform

The SQLite date comparison only appeared in live and loaded runs.

Small interfaces break dependency cycles

By separating "record" from "learn" the Part 2 engine was allowed to run asynchronously without circular dependencies.

Technology Recommendations

Ingest over MQTT with per-device certificates

Recommendation

Have devices publish readings to an MQTT broker over TLS, with the API subscribing from a BackgroundService, instead of posting to POST api/Telemetry. Azure IoT Hub offers the same with a managed broker and per-device X.509 identities.

Justification

MQTT is the standard lightweight publish/subscribe protocol for constrained devices: a persistent connection instead of a request per reading, a header as small as two bytes, and delivery guarantees chosen per message. Its Last Will feature lets the broker announce a device that drops off, which would replace the 15-second silence timer. The load test showed the HTTP path hitting limits first (a 500-reading batch cap that broke the simulator, and about 10 readings a second by default). For security, a certificate per device means only registered hardware can publish; today anyone who can reach the API can post readings for a known MAC address.

.NET fit. MQTTnet is a .NET MQTT client and broker library, Azure IoT Hub has first-party .NET SDKs, and the existing TelemetryPipeline stays as the consumer's queue. Trade-off: a broker to run and secure.

Push changes to the browser with SignalR

Recommendation

Replace the client's 2-second polling with ASP.NET Core SignalR, so the server sends a message when the mesh or an alert changes.

Justification

Polling costs the same whether or not anything changed. At 1,000 devices the topology response is 816 KB (130 KB compressed), requested every 2 seconds by every open page, and making that tolerable took a version parameter, a render guard and compression. The mesh already has a version that changes only when the analysis does, which is exactly the event to push. This improves scale (traffic proportional to changes, not to clients) and performance (the page does work only when something changed).

.NET fit

SignalR is part of ASP.NET Core, uses WebSockets with automatic fallback, and has a client package that runs in Blazor WebAssembly. Trade-off: persistent connections use server memory; keep polling as a fallback.

Partitioned time-series storage, with the AVL tree as a hot cache

Recommendation

Move telemetry to PostgreSQL with time-based partitioning (optionally the TimescaleDB extension), and keep the in-memory AVL trees only for the recent window the live pages query.

Justification

The tree gives microsecond range queries, but it holds everything: at the default 30-day retention, 200 devices need about 5.8 GB of memory and 14.5 GB of disk, and SQLite allows one writer. Partitioning makes retention a cheap operation (dropping a

partition instead of deleting millions of rows), serves range queries from a time index, and allows old readings to be downsampled. This is a scale recommendation; it is not worth the extra server below roughly 100 devices.

.NET fit

The Npgsql provider plugs into EF Core, so the existing AppDbContext and LINQ queries carry over.

Authenticate and authorise the API

Recommendation

Add JWT bearer or OpenID Connect authentication (for example Microsoft Entra ID), role policies (operator and viewer) on the controllers, and ASP.NET Core's built-in rate limiting.

Justification

This is a real gap today: the command endpoints that switch actuators on and off, undo commands and register or delete sensors are open to anyone who can reach the API, and CORS only restricts browsers. On a vineyard, an unauthorised caller could shut an irrigation valve. Rate limiting also protects the ingest endpoint from floods. This is the security recommendation.

.NET fit

AddAuthentication().AddJwtBearer(), [Authorize(Policy = ...)] and AddRateLimiter() are built into ASP.NET Core, so no new framework is needed.

Observability and automated checks (OpenTelemetry and CI)

Recommendation

Export metrics with OpenTelemetry (ingest queue depth, analysis time, response size, database log size) and run a GitHub Actions workflow that builds with warnings as errors, runs the test scripts and runs a short load test.

Justification

The unbounded SQLite log was found only because I looked at the files after a load run; a metric for the log size would have shown it as it grew. A pipeline that must compile and pass the tests matches the rule that code that does not run earns nothing, and the load test would catch regressions such as the simulator's 500-device limit before release. This improves reliability and scale in operation.

.NET fit

.NET's System.Diagnostics.Metrics is the native metrics API, and the OpenTelemetry .NET SDK and ASP.NET Core instrumentation export it to Prometheus or Azure Monitor.

References

Adelson-Velsky, G.M. and Landis, E.M. (1962) ‘An algorithm for the organization of information’, *Doklady Akademii Nauk SSSR*, 146(2), pp. 263–266.

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) *Introduction to algorithms*. 4th edn. Cambridge, MA: MIT Press.

Dijkstra, E.W. (1959) ‘A note on two problems in connexion with graphs’, *Numerische Mathematik*, 1(1), pp. 269–271. Available at: https://doi.org/10.1007/BF01386390 (Accessed: 13 November 2026).

GitHub (n.d.) *GitHub Actions documentation*. Available at: https://docs.github.com/en/actions (Accessed: 13 November 2026).

Heinzelman, W.R., Chandrakasan, A. and Balakrishnan, H. (2000) ‘Energy-efficient communication protocol for wireless microsensor networks’, *Proceedings of the 33rd Annual Hawaii International Conference on System Sciences*. Maui, HI, 4–7 January. IEEE. Available at: https://doi.org/10.1109/HICSS.2000.926982 (Accessed: 13 November 2026).

Kruskal, J.B. (1956) ‘On the shortest spanning subtree of a graph and the traveling salesman problem’, *Proceedings of the American Mathematical Society*, 7(1), pp. 48–50. Available at: https://doi.org/10.1090/S0002-9939-1956-0078686-7 (Accessed: 13 November 2026).

Microsoft (2023) *.NET metrics overview*. Available at: https://learn.microsoft.com/en-us/dotnet/core/diagnostics/metrics (Accessed: 13 November 2026).

Microsoft (2025) *Authenticate with X.509 certificates: Azure IoT Hub*. Available at: https://learn.microsoft.com/en-us/azure/iot-hub/authenticate-authorize-x509 (Accessed: 13 November 2026).

Microsoft (2026a) *Overview of ASP.NET Core authentication*. Available at: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/ (Accessed: 13 November 2026).

Microsoft (2026b) *Overview of ASP.NET Core SignalR*. Available at: https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction (Accessed: 13 November 2026).

Microsoft (2026c) *Rate limiting middleware in ASP.NET Core*. Available at: https://learn.microsoft.com/en-us/aspnet/core/performance/rate-limit (Accessed: 13 November 2026).

Microsoft (2026d) *SQLite database provider: limitations: EF Core*. Available at: https://learn.microsoft.com/en-us/ef/core/providers/sqlite/limitations (Accessed: 13 November 2026).

MQTTnet (n.d.) *MQTTnet: a .NET library for MQTT*. Available at: https://github.com/dotnet/MQTTnet (Accessed: 13 November 2026).

Npgsql (n.d.) *Npgsql Entity Framework Core provider for PostgreSQL*. Available at: https://www.npgsql.org/efcore/ (Accessed: 13 November 2026).

OASIS (2019) *MQTT version 5.0: OASIS standard*. Available at: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html (Accessed: 13 November 2026).

OpenTelemetry (n.d.) *OpenTelemetry for .NET*. Available at: https://opentelemetry.io/docs/languages/dotnet/ (Accessed: 13 November 2026).

PostgreSQL Global Development Group (n.d.) *Table partitioning*. Available at: https://www.postgresql.org/docs/current/ddl-partitioning.html (Accessed: 13 November 2026).

Prim, R.C. (1957) 'Shortest connection networks and some generalizations', *Bell System Technical Journal*, 36(6), pp. 1389–1401. Available at: https://doi.org/10.1002/j.1538-7305.1957.tb01515.x (Accessed: 13 November 2026).

SQLite (n.d.) *Write-ahead logging*. Available at: https://www.sqlite.org/wal.html (Accessed: 13 November 2026).